

Algorithm Analysis, Searching & Sorting Guide

Comprehensive quick-prep reference including theoretical principles, asymptotic definitions, visual dry-runs, and complete C code implementations.

1. Algorithm Basics & Performance Analysis

An **algorithm** is an unambiguous, finite sequence of computational steps taking input and producing output. Every valid algorithm satisfies five core criteria: *Input* (0 or more), *Output* (at least 1), *Definiteness* (each instruction is clear and unambiguous), *Finiteness* (terminates after finite steps), and *Effectiveness* (operations are practically feasible).

Asymptotic Bounds (Definitions)

Big-O (O) - Upper Bound: Worst-case running time guarantee. $f(n) \leq c \times g(n)$ for all $n \geq n_0$.

Big-Omega (Ω) - Lower Bound: Best-case minimum threshold. $f(n) \geq c \times g(n)$ for all $n \geq n_0$.

Big-Theta (Θ) - Tight Bound: Sandwich bound where $c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$.

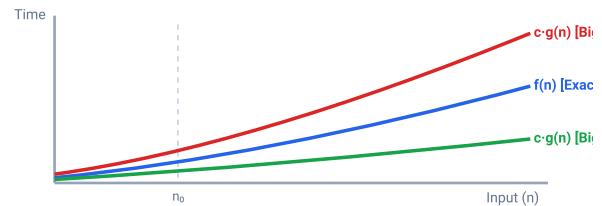


Fig 1: Mathematical relationships between O (ceiling) and Ω (floor).

Master Complexity Reference Sheet

Algorithm	Best Time	Avg Time	Worst Time	Space	Stable?	Core Algorithmic Technique
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	N/A	Sequential examination of every element
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	N/A	Halving sorted candidate range on each step
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Repeatedly select minimum and swap to front
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Shift larger items right to insert key in order
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Divide array into halves, recursively merge
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Partition elements around pivot value

2. Searching Algorithms & Visual Execution

Searching is the process of locating a specific target key within a collection of records. **Linear Search** works on any arbitrary list without preconditions, while **Binary Search** strictly requires the data to be ordered monotonically.

Binary Search Trace: Target = 25 in Sorted Array

Step 1 (L=0, R=5, M=2):	6	12	23 (mid)	25	30	45	23 < 25 → Search right half (L = mid + 1)
Step 2 (L=3, R=5, M=4):	[eliminated]		25	30 (mid)	45		30 > 25 → Search left half (R = mid - 1)
Step 3 (L=3, R=3, M=3):			25	[eliminated]			arr[3] == 25 → Found at index 3!

Linear Search (Inside main)

Iterates sequentially; terminates immediately when target is matched.

```
#include <stdio.h>

int main() {
    int arr[] = {12, 45, 23, 6, 25, 30};
    int n = 6, target = 25, index = -1;

    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            index = i;
            break;
        }
    }

    if (index != -1)
        printf("Found at index %d\n", index);
    else
        printf("Not found\n");
    return 0;
}
```

Binary Search (Inside main)

Requires sorted array; cuts range in half using midpoint test.

```
#include <stdio.h>

int main() {
    int arr[] = {6, 12, 23, 25, 30, 45};
    int n = 6, target = 25;
    int low = 0, high = n - 1, index = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            index = mid;
            break;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (index != -1)
        printf("Found at index %d\n", index);
    return 0;
}
```

3. Elementary Iterative Sorting (In main)

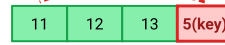
Elementary sorts execute without recursion. **Selection Sort** minimizes writes by conducting exactly $n - 1$ swaps throughout execution, while **Insertion Sort** achieves linear $O(n)$ best-case performance when input lists are already or nearly sorted.

Selection Sort Pass (Scan for min)



Scan remaining unsorted items for min (22), then 1 single swap.

Insertion Sort (Card insertion into sorted prefix)



Key 5 shifts elements {13, 12, 11} right, then drops into index 0.

Selection Sort (Inside main)

Identifies smallest remaining item and performs 1 swap per pass.

```
#include <stdio.h>

int main() {
    int arr[] = {64, 25, 12, 22, 11, 90};
    int n = 6;

    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j;
        }
        int temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }

    printf("Sorted: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Insertion Sort (Inside main)

Picks current element as key and shifts larger items right.

```
#include <stdio.h>

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = 6;

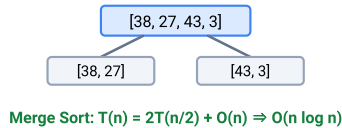
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }

    printf("Sorted: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

4. Divide & Conquer Sorting (Recursive Functions)

Recursive algorithms break problems down into smaller self-similar subproblems. **Merge Sort** guarantees $O(n \log n)$ by merging sorted lists using $O(n)$ auxiliary space. **Quick Sort** partitions in-place around a pivot, giving an average runtime of $O(n \log n)$.

Divide-and-Conquer Workflows



Arr: [10, 80, 30, 90, 40 | Pivot: 50]

Elements < 50

[50]

Elements > 50

Quick Sort: Worst case $O(n^2)$ if pivot is extreme

Merge Sort (Separate Functions)

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82};
    mergeSort(arr, 0, 5);
    for (int i = 0; i < 6; i++) printf("%d ", arr[i]);
    return 0;
}
```

Quick Sort (Separate Functions)

```
#include <stdio.h>

int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
        int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
        return i + 1;
    }
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[] = {10, 80, 30, 90, 40, 50};
    quickSort(arr, 0, 5);
    for (int i = 0; i < 6; i++) printf("%d ", arr[i]);
    return 0;
}
```